Agent-Oriented Architecture

A four-session hands-on workshop — handout

One pattern, four passes

Pass	Session job	What changes
1. **See** the shape	Make AOA visible through small concrete demos	Small workflow
2. **Name** the shape	Inspect the running parts in AOA vocabulary	Running system
3. **Recognise** the shape	See the architecture in the wider landscape	Real landscape
4. **Apply** the shape	Turn the architecture into a contextual adoption loop	Adoption loop

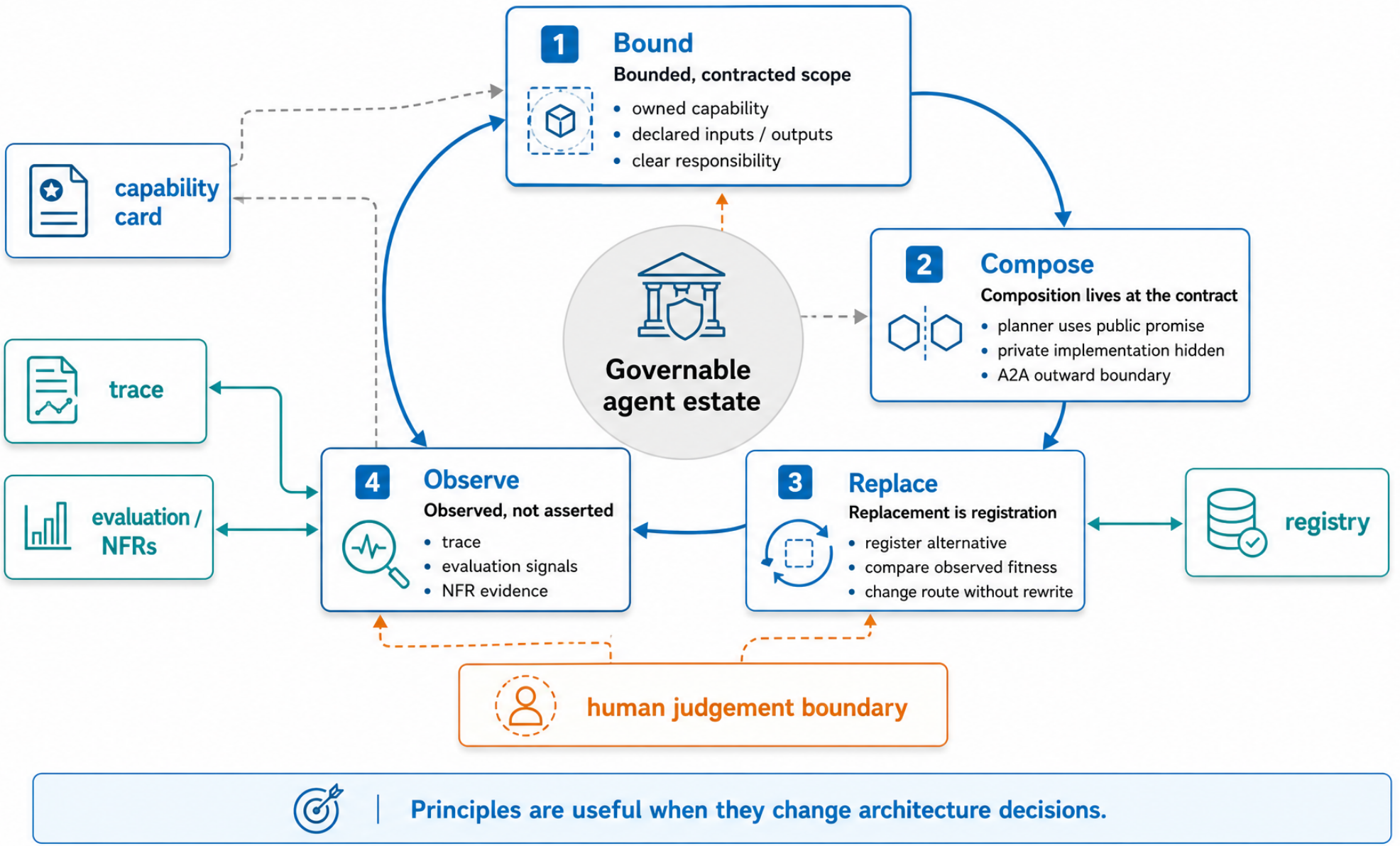
The lifecycle, eight verbs

`describe → discover → compose → execute → observe → evaluate → replace → govern`

Same eight verbs in every session. Different depth each pass.

The four AOA principles (full detail overleaf)

`bounded, contracted scope` · `composition lives at the contract` · `replacement is registration` · `observed, not asserted`


1
Bound
Bounded, contracted scope
owned capability
declared inputs / outputs
clear responsibility
capability card
2
Compose
Composition lives at the contract
planner uses public promise
private implementation hidden
A2A outward boundary
Governable agent estate
trace
evaluation / NFRs
4
Observe
Observed, not asserted
trace
evaluation signals
NFR evidence
3
Replace
Replacement is registration
register alternative
compare observed fitness
change route without rewrite
registry
human judgement boundary
Principles are useful when they change architecture decisions.

The four AOA principles

The load-bearing architectural commitments. Each one is architectural (not operational), falsifiable, independent of the others, atomic, and earns its rent.

1. Bounded, contracted scope

Each Agentic Unit owns a narrow capability with declared inputs, outputs, constraints, and responsibility — a public contract on a capability card. Scope is bounded; unbounded AUs are not AUs.

Decompose your agent estate into units you can actually govern — not frameworks you have to maintain.

2. Composition lives at the contract

Work is composed by reading public capability cards through the registry, not by lines of code that hard-wire one agent to another. The planner reasons over declared promises and constraints at runtime.

Integration without picking a winner. Your existing platform bets stay in place; AOA is what makes them add up.

3. Replacement is registration, not a rewrite

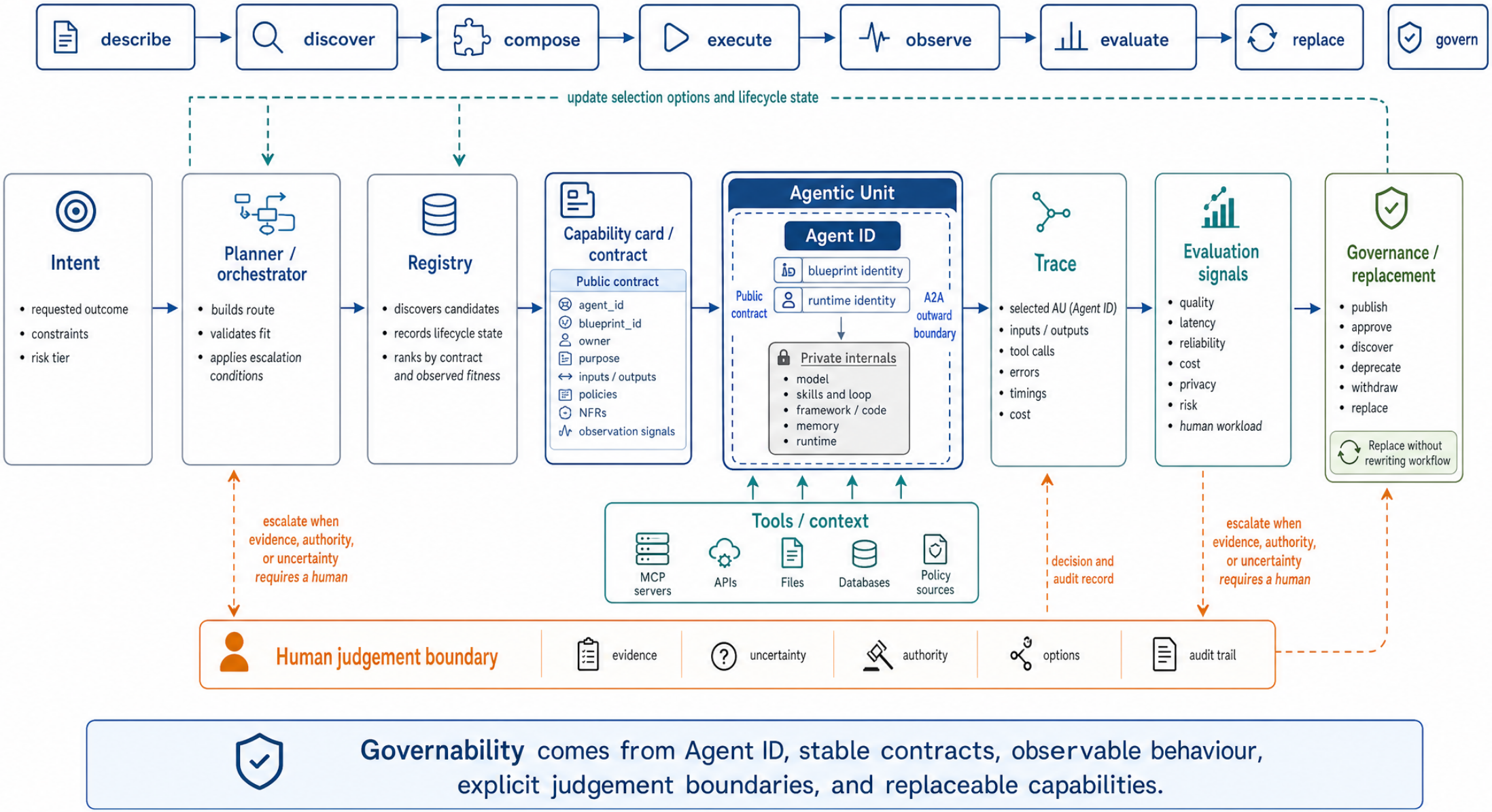
Substituting one AU for another — different backend, model, vendor, or implementation — is a registry operation. Callers don't change; code doesn't change; the planner picks up the new candidate on the next selection.

Architected to outlive the model of the month. No more 18-month re-platforms when the next thing drops.

4. Observed, not asserted

A capability card is a claim; the trace is evidence. Once traffic flows, runtime measurements override the declarations. Selection uses what the system has measured, not what the AU claims. (The architectural reply to UDDI's failure mode.)

Run it like an estate, not like a pilot. Cost, latency, and quality become first-class signals — including something you can take to your CFO.



Governability comes from Agent ID, stable contracts, observable behaviour, explicit judgement boundaries, and replaceable capabilities.

Session 1 · See the shape

Make AOA visible through small concrete demos, then name the four principles.

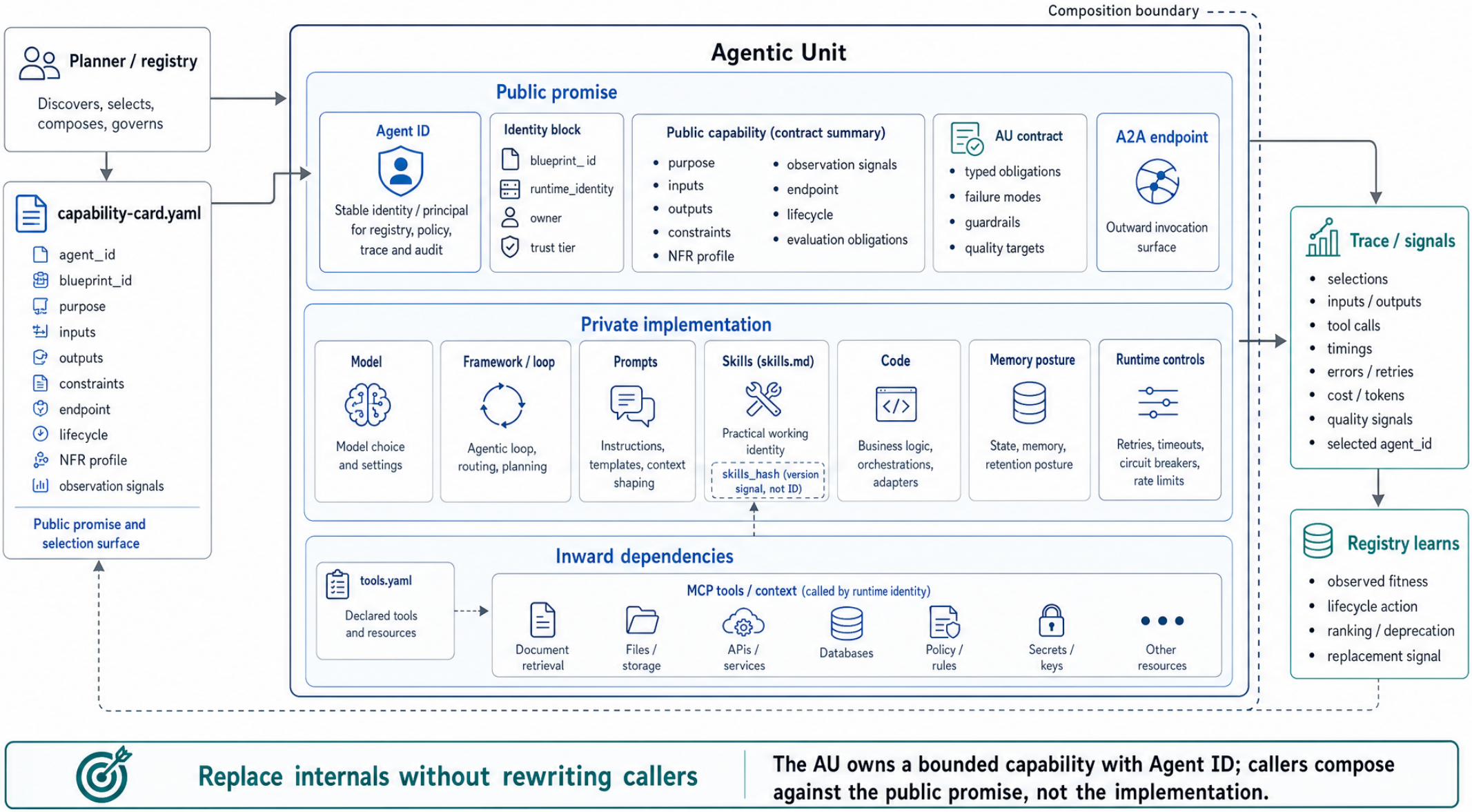
Key notes

- **The agentic inversion.** Workflow composition moves from a human in the loop to a planner reasoning over capability contracts. The unit of value isn't a clever agent — it's a fleet that can be discovered, composed, observed, and replaced.
- **Agent sprawl is the failure mode.** Without a shared catalogue, identity, and governance, duplicate capabilities accumulate, unowned agents drift, and shadow agents proliferate. AOA's principles exist to govern the fleet, not to fight individual agents.
- **Eight key components** make an agent system governable: *AU · capability card · registry · planner · tool boundary · trace · evaluation signal · human oversight point.*
- **Four principles** — bounded scope · composition at the contract · replacement is registration · observed, not asserted. They are architectural constraints, not features of a product.
- **The principles aren't theory.** The expense-report demo shows all four principles working: messy receipts become a policy-checked report through bounded, discovered, observable units — and a small local model carries each step. Local models swap in for frontier models without rewriting the workflow: *replacement is registration* in one demo moment.

Spot the eight key components

During the demo, see if you can spot each one. Pair up after the demo for two minutes and compare notes.

#	Component	What it is
1	Agentic Unit (AU)	a bounded capability — one job
2	Capability card	public contract — inputs / outputs
3	Registry	where cards live + are discoverable
4	Planner	composes work by reading cards
5	Tool boundary	where an AU calls a tool (MCP)
6	Trace	runtime record of what happened
7	Evaluation signal	observed quality — cost, latency, confidence
8	Human oversight point	where a human reviews / approves / escalates



Replace internals without rewriting callers

The AU owns a bounded capability with Agent ID; callers compose against the public promise, not the implementation.

Session 2 · Name the shape

Deep dive on the AU. The control surfaces from Session 1 become inspectable, walkable, and governable.

Key notes

- **AU anatomy.** A public *capability card* (what) wraps private *internals* (how) — model, framework, skills, code, memory, runtime. **Agent ID** is the stable governed identity; hashes are integrity signals, not identity.
- **The capability card is the public promise.** Inputs · outputs · constraints · NFRs · owner · lifecycle state · evaluation evidence · human-oversight policy. The planner reasons over the card; consumers compose against the card.
- **Instructions are governed prose.** The behaviour file is `instructions.md` — markdown read by both humans and the model, bound to a specific capability card. Edits are first-class governance events — recorded as `skills_hash` (the hash keeps its historical name) so observations can be reconciled to the behaviour version that produced them.
- **Tools are not capabilities.** Both can be registered, but a tool is deterministic (MCP-style); an AU has model-driven behaviour. The registry holds both with different `kind`.
- **Trace makes responsibility walkable.** Intent → planner → registry → selected AU → tool calls → outputs → evaluation signals. Correlation IDs tie it together. Without a walkable trace, "observed, not asserted" is a slogan.
- **Silent recovery vs explicit escalation.** Silent recovery is permitted *inside* the contract and *visible in the trace*. Escalate at constraint boundaries — missing capability, irreversible side-effect, trust-tier breach.

The lab you did

You edited `system/agents/evaluator/capabilities/cv/instructions.md` in the running system, watched `skills_hash` re-register with no restart, and reran the workflow. To repeat at home: `./scripts/session1-up.sh`, Studio at `localhost:8080`. The stretch: swap the model provider in `.env` and watch `provenance.model` change behind the same card.

Era (years)	Contract	Discovery	Coordination	Observation
Distributed objects (1989)	IDL	OMG Trading Service	Activation Services	logs
SOA / web services (2000)	WSDL	UDDI	BPEL / ESB	logs / SLAs
Pragmatic REST (2000)	OpenAPI	(informal — lost)	application code	tracing
Microservices (2011)	OpenAPI / protobuf	Eureka / Consul / K8s	Airflow / Temporal	observability
Event-driven / reactive (2015)	Avro / Protobuf	Schema Registry	Kafka choreography	distributed tracing
Cloud-native / serverless (2018)	OpenAPI / gRPC	Service mesh	K8s + Argo	OpenTelemetry
Micro-agents (2024)	**capability card / agent card**	**agent registry**	**planner / orchestrator**	**evals + traces**

current unit: probabilistic capability

Same responsibilities; different unit behind the contract.

Session 3 · Recognise the shape

The same AOA shape in the wider landscape: frameworks, platforms, standards, registries, sovereignty.

Key notes

- **AOA's 25-year lineage.** CORBA → SOA → REST → microservices → event-driven → cloud-native → micro-agents. Architecture absorbs what is proven to work. AOA is continuation, not a break.
- **Frameworks live inside AUs.** LangGraph, CrewAI, AutoGen, EVE — implementation choices, not architectural boundaries. The architectural boundary is the AU contract. EVE is the strongest current interior — durable sessions, HITL, native evals — and still has no card, no registry, no identity: strong P1/P4 substrate, P3 enabler, not a P2 composition mechanism.
- **Three species, different altitude.** Orchestration frameworks (lowest), personal agents (middle), AOA fleets (highest). The right diagnostic question depends on which species you're looking at.
- **A2A outward + MCP inward.** Standards make Session 2's boundary portable across vendors. A2A is at v1.0: three equal bindings (JSON-RPC / gRPC / HTTP+JSON), signed agent cards, per-request auth — and platform implementations commonly lag the spec, so ask which version. Platforms are *pattern sources*, not rivals (see the platforms page overleaf); AOA is principles, not a product.
- **Registry governance is lifecycle, not a phone book.** Publish → approve → discover → observe → deprecate → withdraw. Without governance, the registry mis-leads selection — UDDI's failure mode in the agent era.
- **Sovereignty is layered.** Data · Model · Infrastructure · National policy — four independent axes, each with its own question and failure mode. *Sovereignty washing* is any claim that doesn't say which layer.
- **The trust pressure test.** *Who is acting, on whose authority, and why should another team or organisation trust it?*

The capstone you did

You scaffolded an agent with `npx eve@latest init`, found it invisible to the governed system (no card, no registry entry, no identity, wrong protocol, no provenance), then wrapped it with the `aoa-eve` adapter and watched it register as a first-class AU. The full lab is `system/agents-eve/EXERCISE.md` — repeat it at home with `./scripts/session3-up.sh`. The takeaway table: EVE gives authoring + runtime; AOA adds contract, discovery, governed identity, interop, observed quality.

Platforms are pattern sources

*This page carries the vendor-platform landscape that the live session traded for the EVE capstone. Read every platform with the same question: which AOA responsibilities does it discharge, and which remain yours to design?***

AOA is principles, not a product. Platforms are pattern sources, not rivals — and not a leaderboard.

Platform	Pattern worth extracting	The unresolved AOA question
AWS AgentCore	Managed-stack convergence: runtime, registry, gateway, memory, identity, policy, observability, evaluation in one stack — strong signal these are becoming standard control surfaces	Can selection and replacement stay evidence-led and cross-platform?
Google Gemini Enterprise / Vertex	Three AOA-novel patterns: SPIFFE per-agent IAM identity (each agent a first-class principal); bindings as a first-class registry resource (scoped, policy-gated source→target edges); a gateway that default-blocks unregistered targets — the closest production "registry as control plane"	Uniform policy across MCP / A2A / REST? Binding portability beyond Google?
Azure Foundry + Entra	The split-registry reality: Foundry runtime, Entra Agent Registry (identity), API Center (MCP tools), M365 Agent Store (consumer discovery) — "the registry" may be several coordinated surfaces	Portable beyond the Microsoft boundary? Cross-registry coordination?
Salesforce + MuleSoft	The agent-fabric pattern: catalogue/discovery, brokered orchestration, and gateway-layer policy as distinct layers	Ownership outside the platform estate?
ServiceNow AI Control Tower	Governance front and centre: agents, models, workflows, skills, prompts, datasets as governed assets; the AI Steward role makes lifecycle accountability explicit	Third-party lifecycle? Observed quality?
Camunda / BPM	Process orchestration with agents: deterministic joins, human approvals, audit trails — strong for compliance-heavy workflows	Is composition design-time process wiring, or contract-led discovery and replacement at runtime?

Permission to stay where you are

If your organisation is on AWS, Azure, Salesforce, ServiceNow, or Camunda, the question is not "throw it away." It is: which AOA responsibilities does your platform **discharge**, which does it **hide**, and which do you still have to **design**?

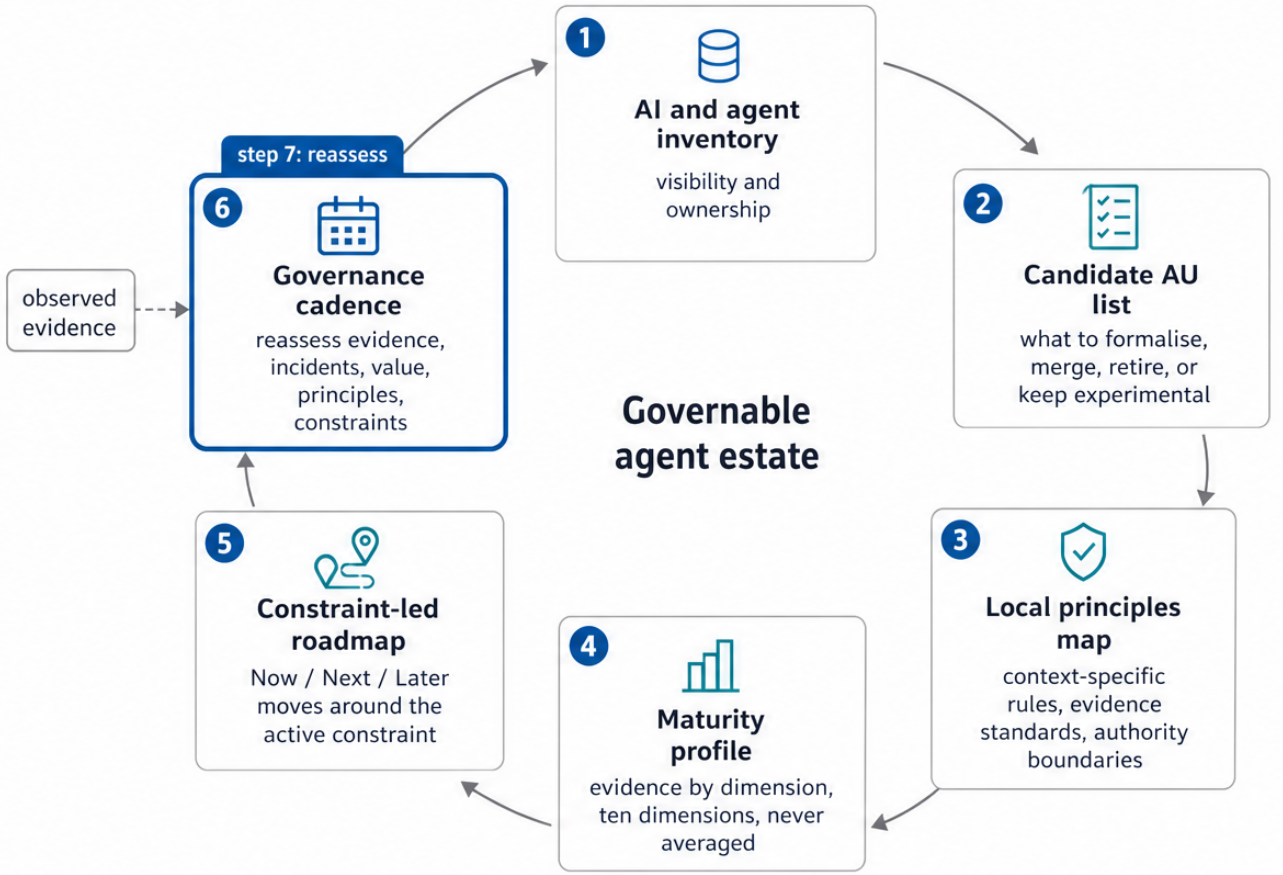
The absorption question

Pattern extraction tells you what a platform discharges. *Absorption* asks how to bring a third-party agent (vendor, partner, OSS) into your estate without inheriting its gaps: a code-level wrapper that normalises shape, a gateway-level proxy that enforces auth / redaction / observability, and an AOA **proxy agent card** registered first-class — declaring the gaps the proxy fills, the skills it refuses to forward, and the trust-class transformation it performs. Without that card, "we proxy our vendors" degenerates into shadow infrastructure the registry cannot reason about. (You built the same seam yourself in the Session 3 capstone — the `aoa-eve` adapter is a proxy pattern in miniature.)

Six adoption questions

1	What exists? →	AI and agent inventory
2	What could become an AU? →	Candidate AU list
3	What rules apply here? →	Local principles map
4	How ready is this boundary? →	Maturity profile
5	What blocks the next safe move? →	Constraint-led roadmap
6	How do we keep it honest? →	Governance cadence

Principles are localised here, then applied to candidate AUs.



Make the organisation more honest before making it more automated.

1 boundary → 2 inventory → 3 principles → 4 maturity → 5 constraint → 6 next move → 7 reassess

Session 4 · Apply the shape

Turn the architecture into a contextual adoption loop you can run on Monday.

Key notes

- **The demo is not step one.** Different starting points need different first moves. AOA is principles, not a product; copy the architecture only where the pressure exists.
- **Maturity is a contextual control loop, not a ladder.** The question is not "*what AOA maturity level are we?*" but "*which capabilities are mature enough for the next adoption move in this context — and which weak dimension is the active constraint?*"
- **The seven-step loop.** *Boundary → inventory → principles → maturity → constraint → next move → reassess.* Run the loop on a specific boundary, not the whole organisation.
- **Profile the boundary, not the org.** Ten dimensions, scored separately, **never averaged**. The weak spoke that the next move requires is the active constraint.
- **Move the active constraint.** *More throughput at the constraint, less WIP, better flow* — not "more agents." Theory of Constraints applied to AOA adoption. Identify → exploit → subordinate → elevate → repeat.
- **Five prioritisation filters: Value · Readiness · Control · Learning · Reuse.** The next move sits where all five overlap. *Now* / *Next* / *Later*, not calendar dates.
- **Six artefacts of a governable agent estate.** AI and agent inventory · AU candidate list · local principles map · maturity profile · constraint-led roadmap · governance cadence.

Monday morning

What to do when the course is over.

The seven-step loop in one sentence

> *Pick a boundary. Inventory what exists. Localise the principles. Build a maturity profile. Find the active constraint. Make the next move. Reassess when the constraint moves.*

The six artefacts of a governable agent estate

1. **AI and agent inventory** — what exists, who owns it, what it touches, lifecycle state.
2. **AU candidate list** — what to formalise, merge, retire, or keep experimental.
3. **Local principles map** — generic AOA principles + your context = your rules of adoption.
4. **Maturity profile** — ten dimensions, scored separately, with evidence and disagreement captured.
5. **Constraint-led roadmap** — Now / Next / Later, around the active constraint, filtered by Value × Readiness × Control × Learning × Reuse.
6. **Governance cadence** — owners review evidence, incidents, value, principles alignment, and whether the constraint has moved.

The honest closing claim

> ***An AOA adoption roadmap should make the organisation more honest before it makes it more automated.*** *It should reveal what AI and agentic capability already exists, where ownership is unclear, which work should be redesigned, which standards are useful, which principles are non-negotiable, and which maturity gaps would make orchestration unsafe.*

Where to go next

- **The running system:** `aoa-course` companion repo at `localhost:8080` — three agents, capability cards, registry view, browser studio with the trace panel.
- **Contact:** `lewis.crawford@equalexperts.com`
- **LinkedIn:** linkedin.com/in/lewiscrawford